HUNDRED DAYS OF CODE
DAY 33

QUESTION: Write a C program to perform the following menu-driven operations using separate functions:

1. Read three variable-length 2D integer arrays A, B, and C, each of dimensions (m x n), and verify the associative property of matrix addition: (A+B)+C=A+(B+C)
2. Read three variable-length 2D integer arrays with the following dimensions: A (m x n), B (n x p), C (p x q). Verify the associative property of matrix multiplication: (A x B) x C = A x (B x C)
3. Exit

SOLUTION:

1. Aim

To write a menu-driven C program using **separate functions and variable-length 2D integer arrays** to verify:

1. The **associative property of matrix addition**: (A + B) + C = A + (B + C)
2. The **associative property of matrix multiplication**: (A × B) × C = A × (B × C)
3. Exit the program.

---

2. Algorithm

Main Algorithm

1. Start the program.
2. Display the menu:
    - Verify associative property of matrix addition.
    - Verify associative property of matrix multiplication.
    - Exit.
3. Accept the user's choice.
4. Use a switch statement to call the corresponding function.
5. Repeat the menu until the user selects **Exit**.
6. Stop.

HUNDRED DAYS OF CODE
DAY 33

Operation 1: Associative Property of Matrix Addition

1. Input the number of rows m and columns n.
2. Declare three variable-length matrices A, B, and C, each of size m × n.
3. Input the elements of all three matrices.
4. Calculate A + B.
5. Add matrix C to obtain (A + B) + C.
6. Calculate B + C.
7. Add matrix A to obtain A + (B + C).
8. Compare the corresponding elements of both resulting matrices.
9. If all elements are equal, display that the associative property of matrix addition is verified.

Operation 2: Associative Property of Matrix Multiplication

Given:

- A is of size m × n
- B is of size n × p
- C is of size p × q

Steps:

1. Input m, n, p, and q.
2. Declare matrices A[m][n], B[n][p], and C[p][q].
3. Input the elements of all three matrices.
4. Calculate A × B, which produces an m × p matrix.
5. Multiply the result by C to obtain (A × B) × C, an m × q matrix.
6. Calculate B × C, which produces an n × q matrix.
7. Multiply A by this result to obtain A × (B × C), an m × q matrix.
8. Compare the two final matrices.

HUNDRED DAYS OF CODE
DAY 33

9. If all corresponding elements are equal, display that the associative property of matrix multiplication is verified.

C PROGRAM:

```
#include <stdio.h>


// Function declarations
void associativeAddition();
void associativeMultiplication();


int main()
{
  int choice;


  do
  {
    printf("\n----- MATRIX OPERATIONS MENU -----\n");
    printf("1. Verify Associative Property of Matrix Addition\n");
    printf("2. Verify Associative Property of Matrix Multiplication\n");
    printf("3. Exit\n");


    printf("Enter your choice: ");
    scanf("%d", &choice);


    switch(choice)
    {
      case 1:
```

HUNDRED DAYS OF CODE
DAY 33

```
            associativeAddition();
            break;

        case 2:
            associativeMultiplication();
            break;

        case 3:
            printf("Program exited successfully.\n");
            break;

        default:
            printf("Invalid choice! Please try again.\n");
      }

   } while(choice != 3);

   return 0;
}



// Function to verify associative property of addition
void associativeAddition()
{
   int m, n, i, j;
   int equal = 1;
```

HUNDRED DAYS OF CODE
DAY 33

```
printf("\nEnter number of rows (m) and columns (n): ");
scanf("%d %d", &m, &n);

// Variable Length Arrays
int A[m][n], B[m][n], C[m][n];
int AB[m][n], BC[m][n];
int left[m][n], right[m][n];

printf("\nEnter elements of Matrix A:\n");
for(i = 0; i < m; i++)
{
   for(j = 0; j < n; j++)
      scanf("%d", &A[i][j]);
}

printf("\nEnter elements of Matrix B:\n");
for(i = 0; i < m; i++)
{
   for(j = 0; j < n; j++)
      scanf("%d", &B[i][j]);
}

printf("\nEnter elements of Matrix C:\n");
for(i = 0; i < m; i++)
{
```

HUNDRED DAYS OF CODE
DAY 33

```
   for(j = 0; j < n; j++)
      scanf("%d", &C[i][j]);
}

// Calculate (A + B) + C and A + (B + C)
for(i = 0; i < m; i++)
{
   for(j = 0; j < n; j++)
   {
      AB[i][j] = A[i][j] + B[i][j];
      left[i][j] = AB[i][j] + C[i][j];

      BC[i][j] = B[i][j] + C[i][j];
      right[i][j] = A[i][j] + BC[i][j];
   }
}

printf("\n(A + B) + C:\n");
for(i = 0; i < m; i++)
{
   for(j = 0; j < n; j++)
      printf("%d\t", left[i][j]);

   printf("\n");
}
```

HUNDRED DAYS OF CODE
DAY 33

```
    printf("\nA + (B + C):\n");
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
            printf("%d\t", right[i][j]);

        printf("\n");
    }

    // Compare both matrices
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
        {
            if(left[i][j] != right[i][j])
                equal = 0;
        }
    }

    if(equal == 1)
    {
        printf("\n(A + B) + C = A + (B + C)\n");
        printf("Hence, associative property of matrix addition is verified.\n");
    }
}
```

HUNDRED DAYS OF CODE
DAY 33

```
// Function to verify associative property of multiplication
void associativeMultiplication()
{
   int m, n, p, q;
   int i, j, k;
   int equal = 1;

   printf("\nEnter m, n, p and q: ");
   scanf("%d %d %d %d", &m, &n, &p, &q);

   // Variable Length Arrays
   int A[m][n];
   int B[n][p];
   int C[p][q];

   int AB[m][p];
   int BC[n][q];

   int left[m][q];
   int right[m][q];

   printf("\nEnter elements of Matrix A (%d x %d):\n", m, n);
   for(i = 0; i < m; i++)
   {
      for(j = 0; j < n; j++)
```

HUNDRED DAYS OF CODE
DAY 33

```
      scanf("%d", &A[i][j]);
}

printf("\nEnter elements of Matrix B (%d x %d):\n", n, p);
for(i = 0; i < n; i++)
{
   for(j = 0; j < p; j++)
      scanf("%d", &B[i][j]);
}

printf("\nEnter elements of Matrix C (%d x %d):\n", p, q);
for(i = 0; i < p; i++)
{
   for(j = 0; j < q; j++)
      scanf("%d", &C[i][j]);
}

// Calculate A x B
for(i = 0; i < m; i++)
{
   for(j = 0; j < p; j++)
   {
      AB[i][j] = 0;

      for(k = 0; k < n; k++)
         AB[i][j] += A[i][k] * B[k][j];
```

HUNDRED DAYS OF CODE
DAY 33

```
  }
}

// Calculate (A x B) x C
for(i = 0; i < m; i++)
{
  for(j = 0; j < q; j++)
  {
    left[i][j] = 0;

    for(k = 0; k < p; k++)
      left[i][j] += AB[i][k] * C[k][j];
  }
}

// Calculate B x C
for(i = 0; i < n; i++)
{
  for(j = 0; j < q; j++)
  {
    BC[i][j] = 0;

    for(k = 0; k < p; k++)
      BC[i][j] += B[i][k] * C[k][j];
  }
}
```

HUNDRED DAYS OF CODE
DAY 33

```
// Calculate A x (B x C)
for(i = 0; i < m; i++)
{
   for(j = 0; j < q; j++)
   {
      right[i][j] = 0;

      for(k = 0; k < n; k++)
         right[i][j] += A[i][k] * BC[k][j];
   }
}

printf("\n(A x B) x C:\n");
for(i = 0; i < m; i++)
{
   for(j = 0; j < q; j++)
      printf("%d\t", left[i][j]);

   printf("\n");
}

printf("\nA x (B x C):\n");
for(i = 0; i < m; i++)
{
   for(j = 0; j < q; j++)
```

HUNDRED DAYS OF CODE
DAY 33

```
            printf("%d\t", right[i][j]);


        printf("\n");
    }


    // Compare both matrices
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < q; j++)
        {
            if(left[i][j] != right[i][j])
                equal = 0;
        }
    }


    if(equal == 1)
    {
        printf("\n(A x B) x C = A x (B x C)\n");
        printf("Hence, associative property of matrix multiplication is verified.\n");
    }
    else
    {
        printf("\nThe resulting matrices are not equal.\n");
    }
}
```

HUNDRED DAYS OF CODE
DAY 33

TEST CASE TABLE:

Test Case	Operation	Dimensions	Expected Result
1	Addition	A, B, C = 2 × 2	(A+B)+C = A+(B+C)
2	Addition	A, B, C = 2 × 3	Associative property verified
3	Addition	A, B, C = 1 × 1	Both results are equal
4	Multiplication	A=`2×2`, B=`2×2`, C=`2×2`	Both products are equal
5	Multiplication	A=`2×3`, B=`3×2`, C=`2×2`	Both final matrices are 2×2 and equal
6	Multiplication	A=`1×2`, B=`2×3`, C=`3×1`	Both final matrices are 1×1 and equal
7	Exit	Choice = 3	Program terminates
8	Invalid choice	Choice = 5	Displays invalid choice message

HUNDRED DAYS OF CODE
DAY 33

COMPILATION AND EXECUTION:

```
┌──(kali㉿kali)-[~/hdoc]
└─$ ./a.out

──── MATRIX OPERATIONS MENU ────
1. Verify Associative Property of Matrix Addition
2. Verify Associative Property of Matrix Multiplication
3. Exit
Enter your choice: 1

Enter number of rows (m) and columns (n): 2 2

Enter elements of Matrix A:
1 2 3 4

Enter elements of Matrix B:
5 6 7 8

Enter elements of Matrix C:
1 1 1 1

(A + B) + C:
7       9
11      13

A + (B + C):
7       9
11      13

(A + B) + C = A + (B + C)
Hence, associative property of matrix addition is verified.
```

```
1. Verify Associative Property of Matrix Addition
2. Verify Associative Property of Matrix Multiplication
3. Exit
Enter your choice: 2

Enter m, n, p and q: 2 2 2 2

Enter elements of Matrix A (2 x 2):
1 2 3 4

Enter elements of Matrix B (2 x 2):
2 0 1 2

Enter elements of Matrix C (2 x 2):
1 2 0 1

(A x B) x C:
4       12
10      28

A x (B x C):
4       12
10      28

(A x B) x C = A x (B x C)
Hence, associative property of matrix multiplication is verified.

──── MATRIX OPERATIONS MENU ────
1. Verify Associative Property of Matrix Addition
2. Verify Associative Property of Matrix Multiplication
3. Exit
Enter your choice: 3
Program exited successfully
```